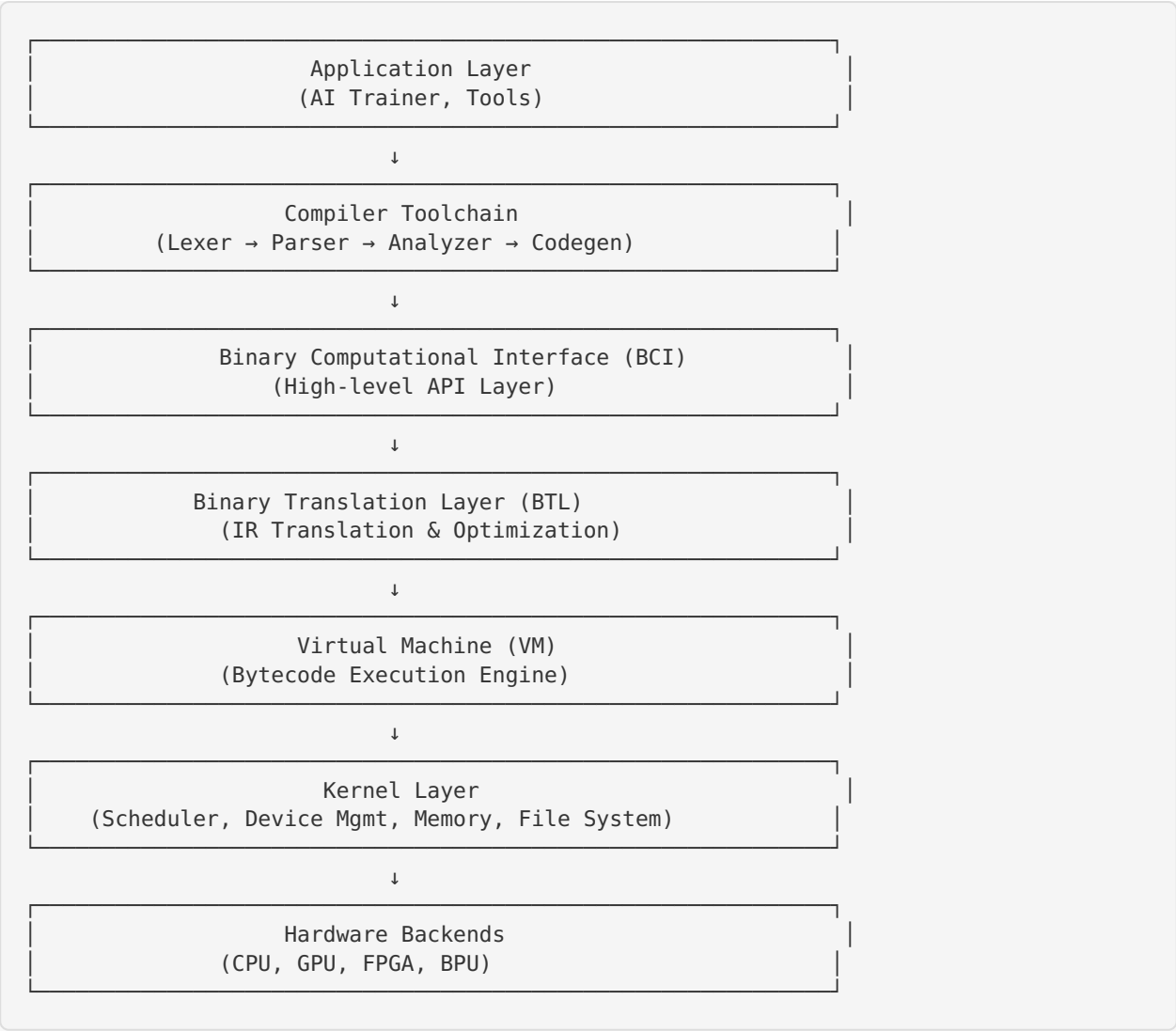


BDI Architecture Documentation

System Overview

The Binary Decomposition Interface (BDI) is a foundational computational substrate designed to represent any computation through a universal fabric. The system is organized into several key layers:



Core Components

1. Binary Computational Interface (BCI)

Location: `C/bci/`

Purpose: Provides the high-level API for interacting with the BDI system.

Key Files:

- `chimera_bci.h/c` : Main BCI interface implementation

Responsibilities:

- API surface for applications

- Request validation and routing
- Resource lifecycle management
- Error handling and reporting

Design Patterns:

- Factory pattern for object creation
- Strategy pattern for backend selection
- Observer pattern for event notifications

2. Binary Translation Layer (BTL)

Location: C/btl/

Purpose: Translates high-level operations into intermediate representation and optimizes execution.

Key Files:

- chimera_btl.h/c : Translation and optimization engine

Responsibilities:

- IR generation from high-level operations
- Optimization passes (constant folding, dead code elimination)
- Target-specific code generation
- JIT compilation support

Optimization Strategies:

- Static analysis for optimization opportunities
- Profile-guided optimization (PGO)
- Link-time optimization (LTO)

3. Compiler Toolchain

Location: C/compiler/

Purpose: Complete compilation pipeline from source to executable representation.

3.1 Lexer

Location: C/compiler/lexer/

Responsibilities:

- Tokenization of source code
- Keyword recognition
- Operator and delimiter handling
- Error reporting for lexical errors

Key Structures:

```
typedef struct Token {
    TokenType type;
    const char* start;
    int length;
    int line;
} Token;
```

3.2 Parser

Location: C/compiler/parser/

Responsibilities:

- Syntax analysis
- AST construction
- Grammar rule enforcement
- Syntax error recovery

Key Structures:

```
typedef struct ASTNode {
    NodeType type;
    struct ASTNode** children;
    int child_count;
    void* data;
} ASTNode;
```

3.3 Semantic Analyzer

Location: C/compiler/semantic_analyzer/

Responsibilities:

- Type checking
- Symbol table management
- Scope resolution
- Semantic error detection

Key Structures:

```
typedef struct Symbol {
    const char* name;
    Type* type;
    Scope* scope;
    bool is_mutable;
} Symbol;
```

3.4 Code Generator

Location: C/codegen/

Responsibilities:

- IR code generation
- Optimization hints
- Target-specific adaptations

4. Virtual Machine (VM)

Location: C/vm/

Purpose: Bytecode execution engine with JIT capabilities.

Key Files:

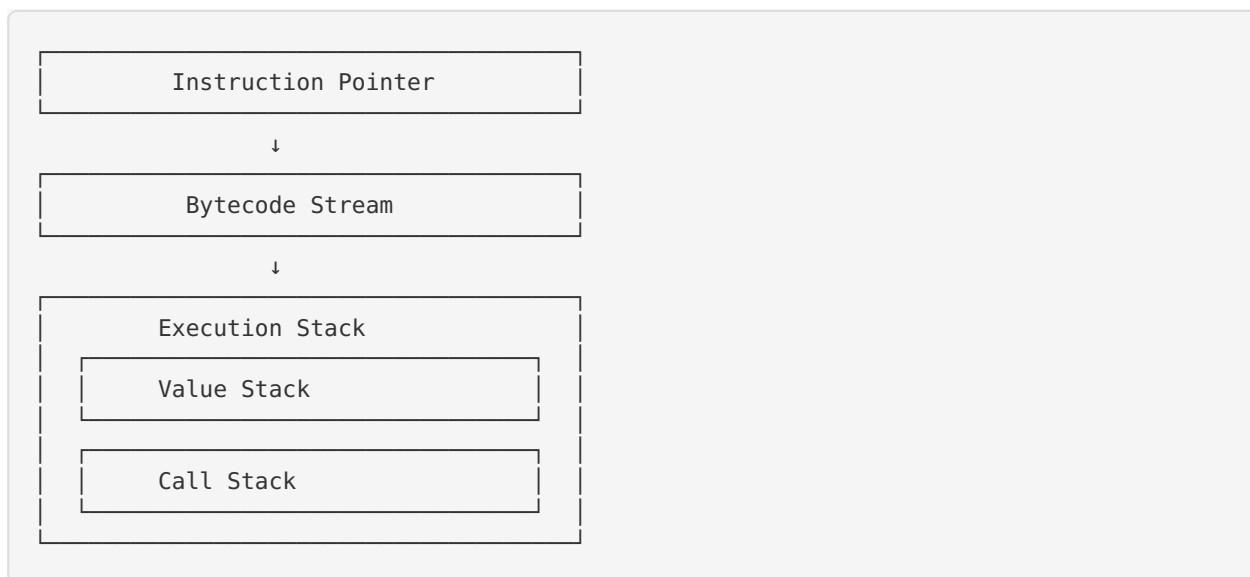
- bci_vm.h/c : VM core implementation
- bci_chunk.h/c : Bytecode chunk management

Responsibilities:

- Bytecode interpretation
- Stack management

- Instruction dispatch
- Runtime type checking

VM Architecture:



5. Kernel Layer

Location: C/kernel/

Purpose: Core system services and resource management.

5.1 Scheduler

Location: C/kernel/scheduler/

Responsibilities:

- Task scheduling and prioritization
- Work queue management
- Load balancing across devices
- Deadline scheduling

Scheduling Algorithm:

- Priority-based scheduling
- Work-stealing for load balancing
- Deadline-aware scheduling for real-time tasks

5.2 Device Management

Location: C/kernel/device/

Responsibilities:

- Device enumeration and initialization
- Device capability querying
- Operation dispatch to devices
- Device-specific optimizations

Supported Devices:

- CPU: Standard execution backend
- GPU: Parallel computation backend

- FPGA: Reconfigurable hardware backend
- BPU: Binary Processing Unit (custom hardware)

5.3 Backend Implementations

Location: C/kernel/backend/

Files:

- gpu_backend.h/c : GPU execution backend
- fpga_backend.h/c : FPGA synthesis and execution
- bpu_device.c : BPU device interface

Backend Interface:

```
typedef struct DeviceBackend {
    int (*init)(void);
    int (*execute)(void** inputs, void** outputs);
    int (*cleanup)(void);
    DeviceCapabilities capabilities;
} DeviceBackend;
```

5.4 File System

Location: C/kernel/file/

Responsibilities:

- File I/O operations
- Buffer cache management
- Transaction logging
- Crash recovery

Key Features:

- Write-ahead logging (WAL)
- Buffer cache for performance
- Atomic operations

5.5 Process Management

Location: C/kernel/process/

Responsibilities:

- Process lifecycle management
- Resource allocation and tracking
- Inter-process communication
- Process isolation

5.6 Graph Execution

Location: C/kernel/graph/

Responsibilities:

- Computational graph representation
- Graph optimization
- Execution planning
- Dependency resolution

5.7 HAM (Hardware Abstraction Module)

Location: C/kernel/ham/

Responsibilities:

- Hardware abstraction layer
- Unified device interface
- Capability negotiation
- Resource virtualization

6. AI Trainer

Location: C/ai_trainer/

Purpose: Machine learning training infrastructure.

Key Files:

- ai_trainer.h/c : Training loop implementation
- ai_trainer_types.h/c : Type definitions for ML operations
- ai_trainer_main.c : Training application entry point

Responsibilities:

- Training loop orchestration
- Gradient computation
- Weight updates
- Model checkpointing

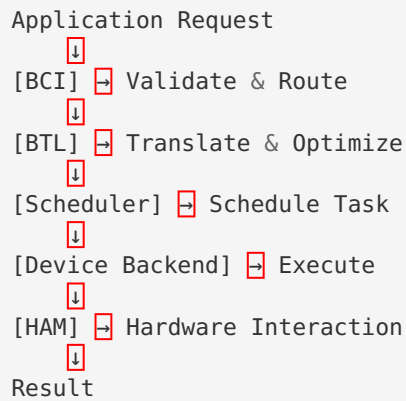
Data Flow

Compilation Flow

```
graph TD; A[Source Code] --> B["[Lexer] → Tokens"]; B --> C["[Parser] → AST"]; C --> D["[Semantic Analyzer] → Validated AST"]; D --> E["[Code Generator] → IR"]; E --> F["[BTL] → Optimized IR"]; F --> G["[VM] → Bytecode"];
```

Source Code
↓
[Lexer] → Tokens
↓
[Parser] → AST
↓
[Semantic Analyzer] → Validated AST
↓
[Code Generator] → IR
↓
[BTL] → Optimized IR
↓
[VM] → Bytecode

Execution Flow



Memory Management

Allocation Strategy

- **Stack:** Local variables, function parameters
- **Heap:** Dynamic allocations via malloc/free
- **Arena:** Bulk allocations for related objects
- **Pool:** Fixed-size object pools for performance

Memory Safety

- Bounds checking in debug builds
- Address sanitizer support
- Memory leak detection
- Use-after-free detection

Concurrency Model

Threading

- Main thread: Application logic
- Worker threads: Task execution
- I/O threads: Asynchronous I/O operations

Synchronization

- Mutexes: Exclusive access to shared resources
- Condition variables: Thread coordination
- Atomic operations: Lock-free data structures
- Read-write locks: Reader-writer scenarios

Lock-Free Structures

- MPSC queues: Multi-producer, single-consumer
- SPSC queues: Single-producer, single-consumer
- Atomic counters: Reference counting

Error Handling

Error Propagation

1. Functions return error codes
2. Errors propagate up call stack
3. Top-level handlers log and recover
4. Critical errors trigger cleanup and exit

Error Categories

- **Recoverable:** Can continue execution
- **Fatal:** Must terminate gracefully
- **Assertion:** Programming errors (debug only)

Performance Considerations

Hot Paths

- Device execution: Minimize overhead
- Scheduler dispatch: Lock-free when possible
- Memory allocation: Use pools for frequent allocations

Optimization Techniques

- Profile-guided optimization (PGO)
- Link-time optimization (LTO)
- Inline critical functions
- Cache-friendly data structures

Profiling Points

- Function entry/exit
- Device operation timing
- Memory allocation tracking
- Lock contention monitoring

Testing Strategy

Unit Tests

- Per-module test suites
- Mock dependencies
- Edge case coverage
- Error path testing

Integration Tests

- Multi-module interactions
- End-to-end workflows
- Performance benchmarks
- Stress testing

Continuous Integration

- Automated builds
- Test execution
- Static analysis
- Code coverage reporting

Future Architecture Considerations

Scalability

- Distributed execution support
- Network-transparent operations
- Cluster scheduling

Extensibility

- Plugin architecture for new backends
- Custom operation support
- User-defined optimizations

Security

- Sandboxed execution
- Resource limits
- Access control
- Audit logging